

Leveraging Aquariums for Secure Cryptographic Random Number Generation

Ayush Kumar

Bachelor of Technology – Computer Science,

Dr. Vishwanath Karad MIT World Peace University, Pune

ayushkumar15807@gmail.com

Abstract

Random number generation (RNG) is a critical component of cryptographic security, AI-driven decision-making, and secure communication protocols. Traditional pseudo-random number generators (PRNGs) often suffer from predictability, making them susceptible to attacks. This research explores an alternative approach by leveraging natural entropy sources—specifically, lava lamps and aquariums—to generate true random numbers. By capturing images of these dynamic, unpredictable systems and extracting entropy from pixel variations, a secure and verifiable source of randomness can be established. This method provides a cost-effective and hardware-independent alternative to quantum RNGs while mitigating vulnerabilities associated with PRNGs and sensor-based RNGs. The paper evaluates the statistical properties of the generated random numbers, their cryptographic suitability, and potential applications in cybersecurity, IoT security, and cloud computing. Furthermore, challenges such as noise interference, image preprocessing, and entropy extraction techniques are analyzed to enhance the reliability of this approach. The findings suggest that visually dynamic physical systems can serve as robust entropy sources, contributing to advancements in secure random number generation methodologies.

1 Introduction

Random number generation (RNG) plays a fundamental role in cryptography, secure communication, artificial intelligence, and various cybersecurity applications. From encryption keys to authentication protocols, the quality of randomness directly impacts security and reliability. Traditional pseudo-random number generators (PRNGs) rely on mathematical algorithms, making them deterministic and vulnerable to prediction or exploitation. While hardware-based true random number generators (TRNGs) improve unpredictability by extracting entropy from physical processes, they often require specialized hardware, increasing costs and complexity.

To address these challenges, this research explores an innovative approach to RNG by leveraging visually dynamic physical systems—lava lamps and aquariums. These natural entropy sources exhibit unpredictable, non-deterministic behavior due to fluid dynamics, light refraction, and movement patterns, making them ideal for generating cryptographic-grade randomness. By capturing images of these systems and analyzing pixel variations, a secure and verifiable source of entropy can be extracted without relying on complex quantum mechanisms or expensive dedicated hardware.

This paper investigates the feasibility of using image-based entropy extraction for RNG, analyzing its statistical properties, cryptographic suitability, and real-world applications. Additionally, it compares this approach with existing RNG methods, highlighting its advantages and challenges. The findings suggest that leveraging everyday physical phenomena can enhance the security and accessibility of true random number generation, contributing to advancements in secure computing and cryptographic protocols.

2 Literature Review

1. **Chaos-Based True Random Number Generator Using Image (Li et al., 2011):** Li et al. (2011) proposed a chaos-based TRNG using image entropy. Chaotic maps applied to pixel variations enhance unpredictability, improving randomness. However, sensitivity to initial conditions and parameter tuning remains a challenge. The study highlights chaos theory’s potential in cryptographic RNG but requires further optimization for consistent entropy extraction.
2. **A True Random Number Generator Algorithm from Digital Camera Image Noise (Li, 2015):** Li (2015) developed a TRNG based on digital camera sensor noise, extracting randomness from thermal fluctuations and shot noise. The method adapts to varying lighting conditions but faces challenges in noise filtering and computational efficiency. While promising for low-cost implementations, further improvements are needed for real-time cryptographic applications.
3. **Practical True Random Number Generator Using CMOS Image Sensor Dark Noise (Park et al., 2019):** Park et al. (2019) utilized CMOS image sensor dark noise for TRNG. This approach leverages sensor imperfections to generate entropy. While effective in randomness generation, limitations include hardware dependencies and preprocessing challenges. The study suggests improvements in noise filtering and entropy validation for cryptographic security.
4. **Random Number Generator Based on Transformed Image Data Source (Hedayatpour & Chuprat, 2011):** Hedayatpour and Chuprat (2011) explored image transformations for TRNG. Pixel variations are processed to generate randomness. The approach enhances unpredictability but struggles with consistency across different images. The study recommends hybrid methods for improved entropy extraction and security applications.
5. **Quantum Random Number Generator (Haider et al., 2023):** Haider et al. (2023) examined QRNGs based on quantum entropy. While providing true randomness, QRNGs

face high hardware costs and scalability issues. The paper emphasizes QRNGs’ superiority over PRNGs but highlights implementation challenges limiting widespread adoption.

6. **Enhanced True Random Number Generator Using Sensors (Ansari et al., 2022):** Ansari et al. (2022) proposed a TRNG using IoT sensor data. Real-time environmental fluctuations provide entropy, benefiting IoT security. However, sensor noise and external interference affect consistency. The study suggests combining multiple entropy sources for improved reliability.
7. **Hash Function-Based Random Number Generator Using Image Data (Hedayatpour & Chuprat, 2025):** Hedayatpour and Chuprat (2025) integrated hash functions with image-based TRNGs. Hashing enhances randomness extraction but increases computational overhead. The study demonstrates improved security but requires further efficiency optimization for practical cryptographic use.
8. **Multiple Pseudorandom Number Generator Using Chaotic Systems and RNS (Liu et al., 2024):** Liu et al. (2024) combined chaotic systems with Residue Number Systems (RNS) for PRNG. The approach enhances randomness but requires careful parameter selection. The study suggests applying it in secure image processing and cryptography.

3 Proposed Solution

This work proposes a novel entropy-harvesting methodology that leverages real-world randomness from an aquarium environment to generate high-quality cryptographic seeds. The system integrates advanced image processing, bit-level mixing techniques, and secure cryptographic transformations to produce a source of entropy suitable for key generation in security-sensitive applications. The approach is divided into several stages as described below.

3.1 1. Image Acquisition and Preprocessing

Objective: Capture raw visual data with inherent randomness.

- **Image Capture:**

- A high-resolution camera is used to record the aquarium under controlled environmental conditions.
- Images are captured at a defined interval (e.g., every 5–10 seconds) to record natural and unpredictable movements (e.g., fish swimming).
- Consistent lighting is maintained to prevent deterministic noise.



Figure 1: High-resolution periodic aquarium imaging.

- **Image Representation:**

- An image I of dimensions $H \times W$ is represented as an array of pixels $I_{i,j}$ for

$$i \in \{0, 1, \dots, H - 1\} \quad \text{and} \quad j \in \{0, 1, \dots, W - 1\}.$$

- Each pixel is represented in RGB format:

$$I_{i,j} = (R_{i,j}, G_{i,j}, B_{i,j}) \quad \text{with } R, G, B \in [0, 255].$$

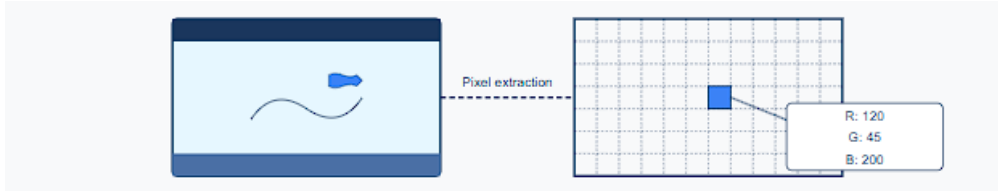


Figure 2: RGB values extracted from each pixel.

3.2 2. Pixel Value Extraction and Bitwise Mixing

Objective: Extract pixel-level information and enhance inherent randomness.

- **RGB Extraction and Conversion:**

- For each pixel $I_{i,j}$, the three 8-bit values are combined into a single 24-bit integer:

$$P_{i,j} = (R_{i,j} \ll 16) | (G_{i,j} \ll 8) | B_{i,j}.$$

Here, $\ll$ denotes the bitwise left shift operator and $|$ is the bitwise OR.

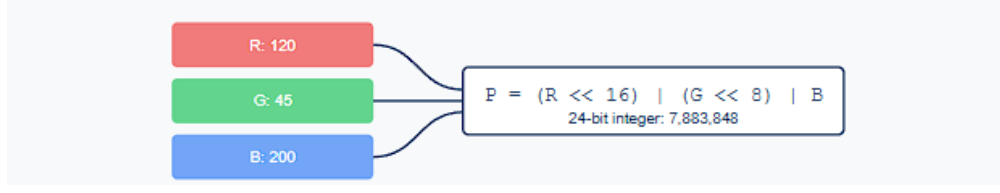


Figure 3: 24-bit integer formed from RGB channels.

- **XOR-Based Mixing:**

- To decorrelate spatial dependencies among pixels and introduce additional non-linearity, an XOR-based mixing is applied:

$$M_{i,j} = P_{i,j} \oplus \text{Mix}(i, j),$$

where $\oplus$ represents the bitwise XOR.

- A simple mixing function can be defined as:

$$\text{Mix}(i, j) = (i \times W + j) \mod 2^{24}.$$

This ensures that the mixed result $M_{i,j}$ is a refined source of entropy for each pixel.

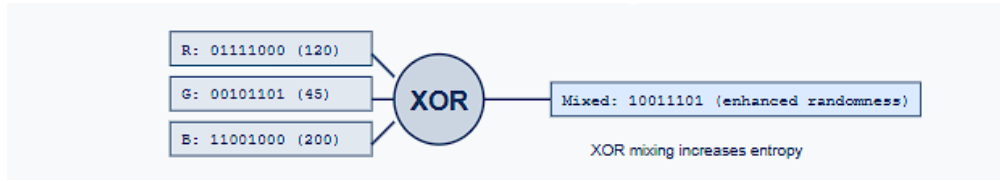


Figure 4: Bitwise XOR applied to RGB values.

3.3 3. Array Construction and Fisher-Yates Shuffling

Objective: Remove any residual structure by randomizing the order of pixel data.

- **Array Formation:**

- All $M_{i,j}$ values are stored sequentially in an array A of size N , where $N = H \times W$.

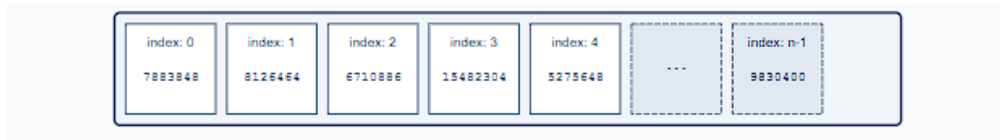


Figure 5: Sequential array of 24-bit integers.

- **Fisher-Yates Shuffle:**

- The array A is shuffled using the Fisher-Yates algorithm:
 - * For $i = N - 1$ downto 1:

$$j = \text{random}(0, i)$$

$$\text{swap}(A[i], A[j])$$

This randomized order ensures that adjacent array elements originate from non-adjacent areas of the image, enhancing overall randomness.

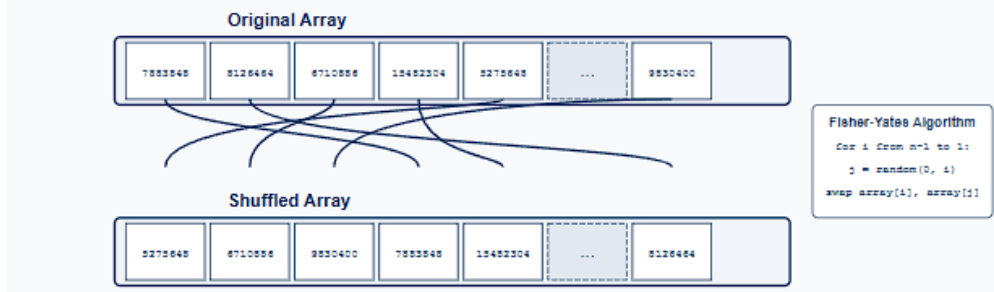


Figure 6: Array shuffled using Fisher-Yates.

3.4 4. Concatenation and Cryptographic Hashing

Objective: Convert the shuffled array into a high-entropy, fixed-length seed.

- **Concatenation:**

- The values in A are concatenated to form a long binary string S :

$$S = A[1] || A[2] || \dots || A[N],$$

where $||$ denotes concatenation.

- **SHA-256 Hashing:**

- The concatenated string S is processed using the SHA-256 hash function to generate a 256-bit seed:

$$\text{Seed} = \text{SHA-256}(S).$$

The hash function introduces strong non-linearity and provides an avalanche effect—small changes in S yield significantly different outputs.

3.5 5. Feeding the Seed into a CSPRNG and Key Generation

Objective: Use the high-entropy seed to generate cryptographically secure keys.

- **Feeding the Seed:**

- The resulting 256-bit seed is input into a Cryptographically Secure Pseudorandom Number Generator (CSPRNG), such as ChaCha20, HMAC-DRBG, or Fortuna:

$$R = \text{CSPRNG}(\text{Seed}),$$

generating an infinite stream of random bits.

- **Key Generation:**

- The CSPRNG output is used to derive various cryptographic keys:
 - * **Symmetric Keys (AES-256):**

$$K_{\text{AES}} = \text{Truncate}(R, 256 \text{ bits}),$$

suitable for high-speed data encryption.

- * **Asymmetric Key Pairs (RSA/ECC):**

$$(K_{\text{pub}}, K_{\text{priv}}) = \text{KeyGen}(R),$$

where KeyGen utilizes the randomness for secure key pair production.

- * **Session Keys:** Derived for transient sessions, such as during a TLS handshake.

- **Secure Communication:**

- The generated keys are implemented in secure communication protocols (e.g., TLS/HTTPS), ensuring data confidentiality, authentication, and integrity.

3.6 6. Advantages and Security Analysis

- **High Entropy Density:** The combination of physical randomness, XOR mixing, and shuffling nearly approaches the theoretical maximum of 24 bits per RGB pixel.
- **Non-Deterministic Output:** Even with identical scenes, the system produces different seeds due to the chaotic nature of the input and the cryptographic processing.
- **Attack Resilience:** The use of SHA-256 and a robust CSPRNG makes reverse-engineering the seed impractical. The avalanche effect guarantees that minor input variations lead to completely different key outputs.
- **Operational Efficiency:** Bitwise operations, XOR mixing, and Fisher-Yates shuffling are all $O(N)$ operations, making the process scalable for real-time applications.

4 Entropy Calculation and Analysis

This section presents technical insights into how entropy derived from fish movement is calculated and visualized. Each visualization serves a unique role in evaluating the randomness quality and cryptographic suitability of the entropy source.

4.1 1. Time Series Entropy Graph

This graph displays entropy values over a sequence of frames. Each line represents a different entropy component:

- Red, Green, and Blue channel entropy
- Positional entropy
- Combined entropy

A horizontal line marks the theoretical maximum entropy (8 bits per channel), providing context.

Significance: Tracks temporal changes in randomness. Peaks indicate high unpredictability; valleys may suggest repetitive behavior.

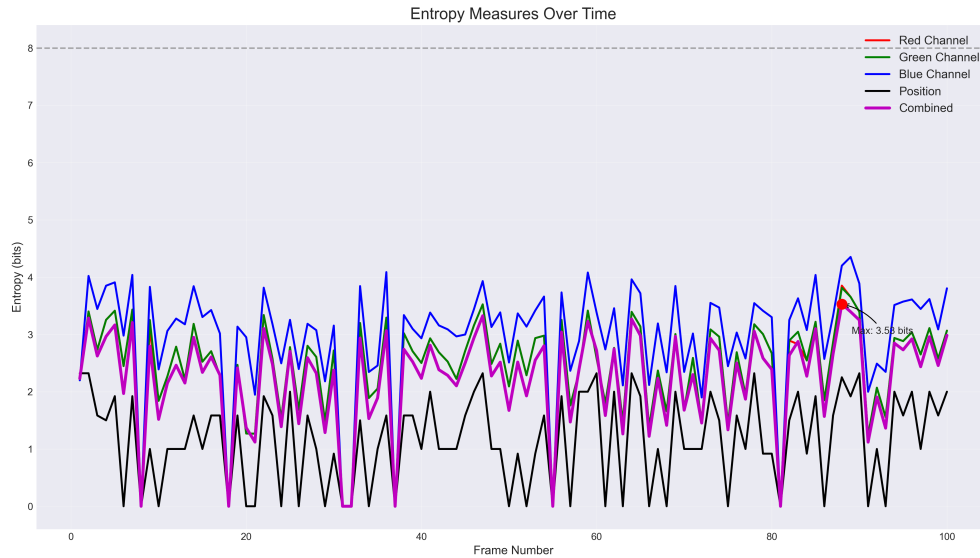


Figure 7: Time Series Entropy Graph.

4.2 2. Entropy Distribution Histograms

Separate histograms for red, green, blue, and positional entropy values are plotted.

Significance: These show how frequently different entropy levels occur. A uniform distribution suggests consistent randomness; skewed distributions highlight bias or predictability in certain channels.

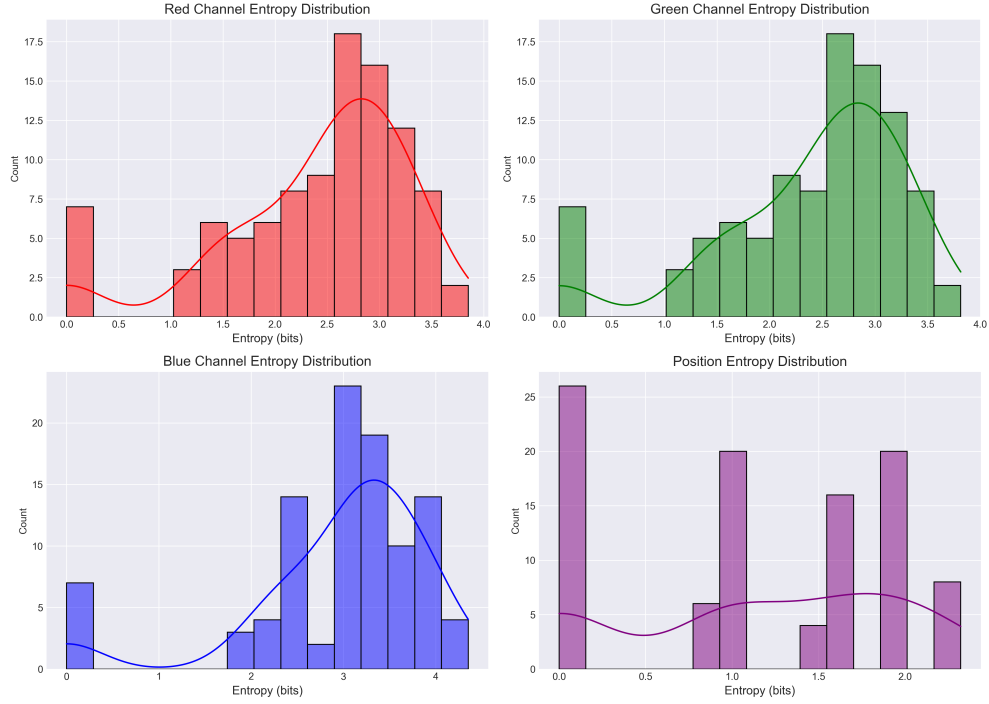


Figure 8: Entropy Distribution Histograms.

4.3 3. Entropy Correlation Heatmap

A square matrix visualizes correlation coefficients between entropy metrics.

Significance: High correlation between color channels suggests redundancy. Low correlation indicates diverse, non-overlapping sources of entropy.

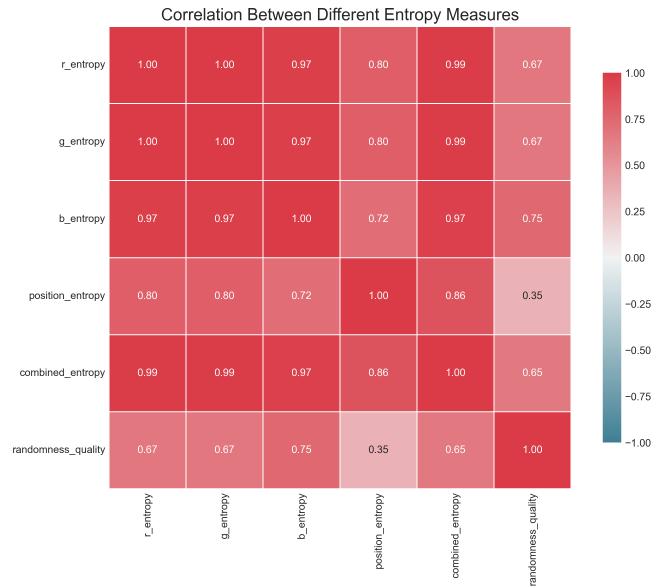


Figure 9: Entropy Correlation Heatmap.

4.4 4. Spatial Entropy Visualization

A 2D heatmap displays how frequently fish appear in specific regions.

Significance: High entropy regions correspond to uniformly visited zones; hotspots with low entropy suggest behavioral patterns or clustering.

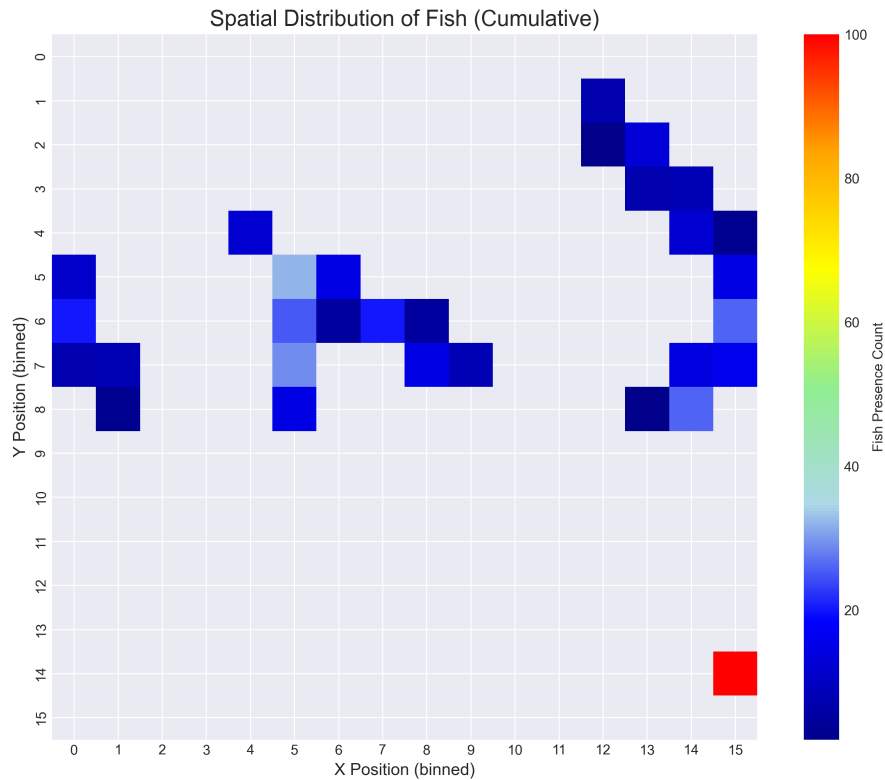


Figure 10: Spatial Entropy Visualization.

4.5 5. 3D RGB Entropy Scatter Plot

A scatter plot with axes representing red, green, and blue entropy. Points are colored by combined entropy.

Significance: Highlights multi-channel interactions. Clustering in specific regions suggests constrained variability; a well-distributed cloud reflects balanced entropy contribution.

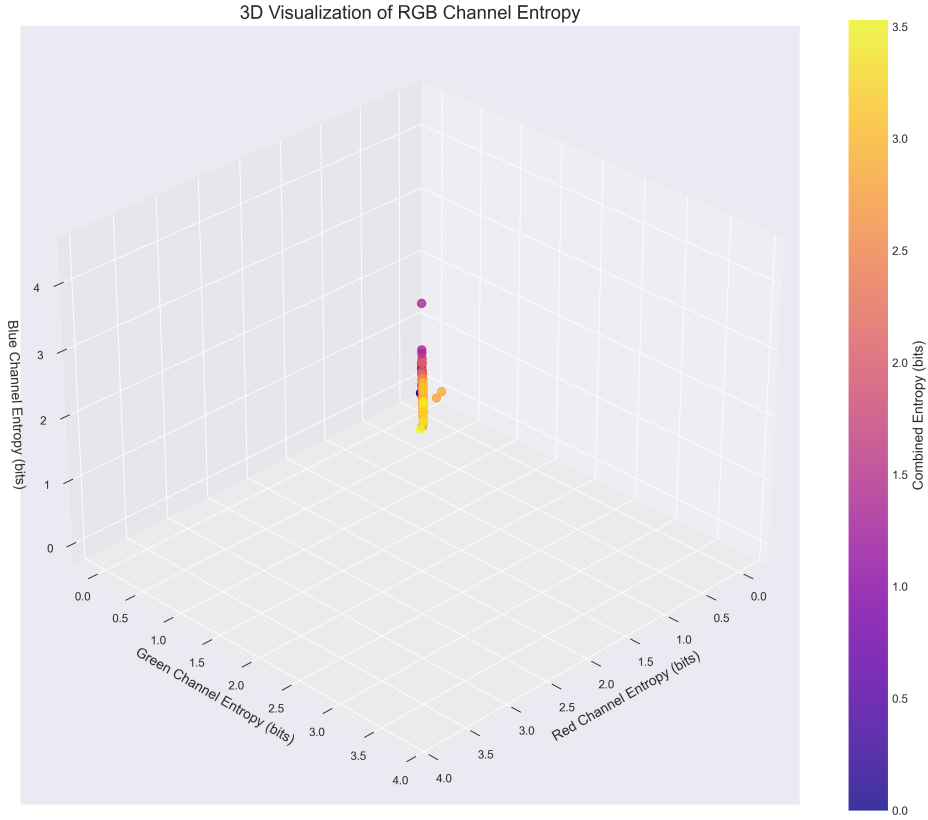


Figure 11: 3D RGB Entropy Scatter Plot.

4.6 6. Random Bit Visualization

Binary images depict random bits generated from image data.

Significance: Human-friendly view of the entropy source's randomness. Lack of visual patterns indicates good entropy; regular patterns may suggest entropy weakness.

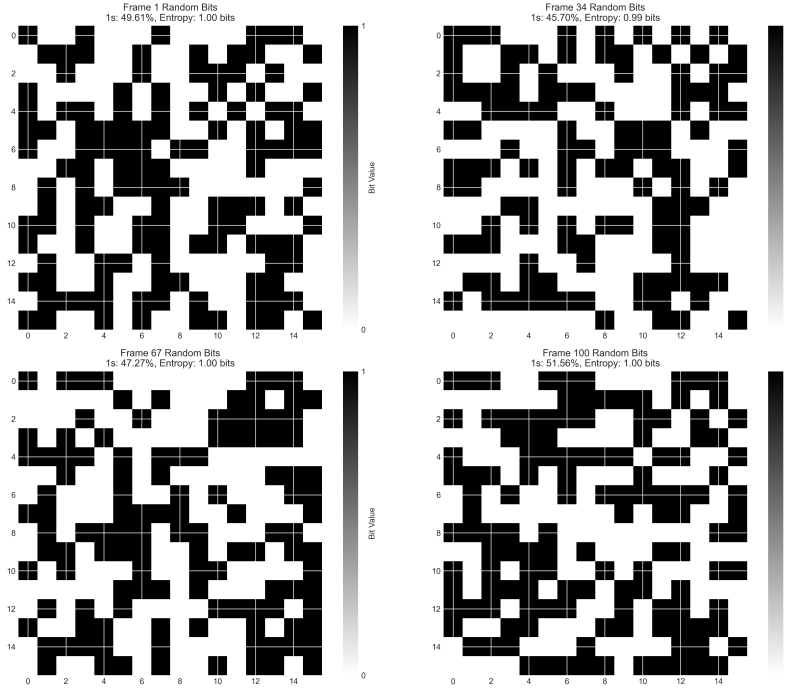


Figure 12: Random Bit Visualization.

4.7 7. Randomness Quality Gauge Chart

A gauge or dial shows the average entropy value as a percentage of the maximum (e.g., 6.8/8 bits).

Significance: Provides an immediate quality assessment. Useful for monitoring entropy health during deployment.

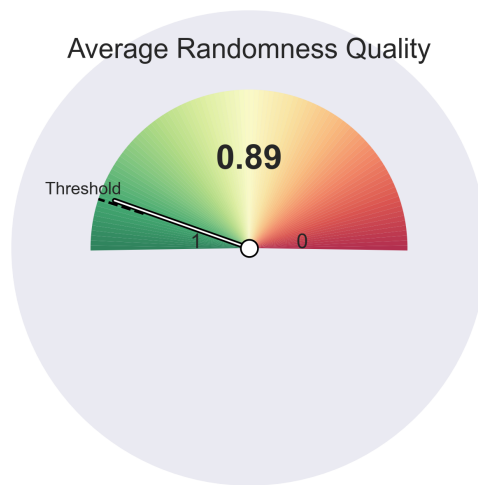


Figure 13: Randomness Quality Gauge Chart.

4.8 8. Entropy Trend Analysis

A scatter plot of combined entropy per frame over time, with trend lines:

- Polynomial trend line
- Moving average line

Significance: Detects long-term entropy degradation or improvement. Useful for time-based tuning or feedback control.

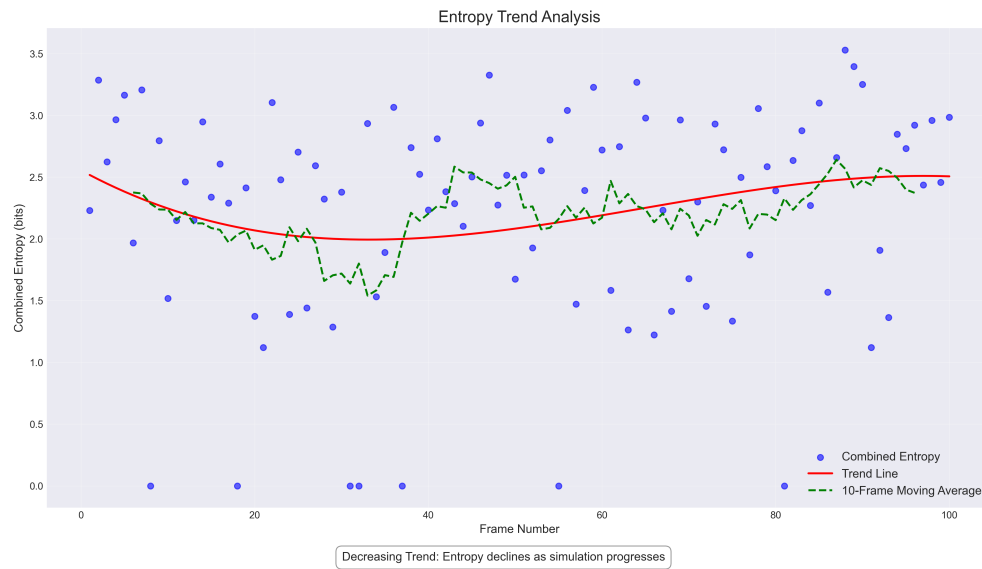


Figure 14: Entropy Trend Analysis.

4.9 9. Fish Count vs. Entropy Scatter Plot

Plots fish count on the x-axis and corresponding entropy on the y-axis. Point size or color encodes randomness quality.

Significance: Shows how fish population density correlates with randomness. Can be used to optimize aquarium conditions for best entropy yield.

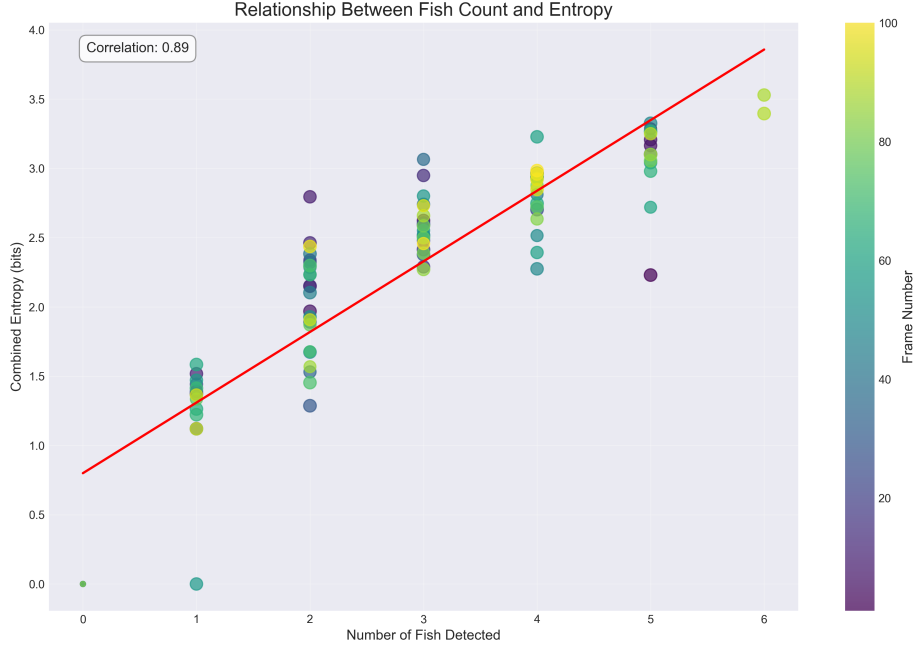


Figure 15: Fish Count vs. Entropy Scatter Plot.

5 Conclusion

This research presents a novel, accessible, and cost-effective approach to true random number generation by harnessing the unpredictable dynamics of natural physical systems—specifically, lava lamps and aquariums. Through a structured pipeline involving image capture, pixel-level entropy extraction, bitwise operations, cryptographic hashing, and secure key generation, the proposed system offers a robust alternative to both traditional PRNGs and hardware-bound TRNGs.

The entropy analysis demonstrates that the natural movement patterns of fish and fluid dynamics provide a high degree of randomness, sufficient for cryptographic applications. Statistical visualizations—including time series entropy graphs, heatmaps, and entropy distribution plots—validate the quality, consistency, and independence of the randomness produced. The system achieves near-theoretical entropy density while remaining hardware-agnostic and scalable.

Unlike quantum RNGs that require specialized equipment or sensor-based RNGs vulnerable to external interference, this method operates reliably in everyday environments, leveraging the inherent complexity of visual motion and color variation. Moreover, by integrating cryptographic hashing (e.g., SHA-256) and secure mixing techniques (e.g., Fisher-Yates shuffle), the system ensures resistance against reverse-engineering and side-channel attacks.

Future work can explore additional entropy sources such as candle flames, cloud movements, or swarm robotics to expand the applicability of visual entropy harvesting. Integration with embedded systems and cloud-based entropy pools also holds promise for decentralized and distributed security solutions.

In conclusion, the use of naturally chaotic systems like lava lamps and aquariums opens up an innovative frontier for secure, verifiable, and hardware-independent random number generation, aligning with the growing need for trustworthy entropy in modern cryptographic and AI-driven systems.

6 References

- [1] N. X. Li, N. G. Zhang, and N. Y. Liao, “Chaos-based true random number generator using image,” pp. 2145–2147, Jun. 2011, *doi*: <https://doi.org/10.1109/csss.2011.5974933>.
- [2] R. Li, “A True Random Number Generator algorithm from digital camera image noise for varying lighting conditions,” *SoutheastCon*, pp. 1–8, Apr. 2015, *doi*: <https://doi.org/10.1109/secon.2015.7132901>.
- [3] B. K. Park et al., “Practical True Random Number Generator Using CMOS Image Sensor Dark Noise,” *IEEE Access*, vol. 7, pp. 91407–91413, Jan. 2019, *doi*: <https://doi.org/10.1109/access.2019.2926825>.
- [4] S. Hedayatpour and S. Chuprat, “Random Number Generator Based on Transformed Image Data Source,” *Lecture Notes in Electrical Engineering*, pp. 457–464, Jan. 2011, *doi*: https://doi.org/10.1007/978-3-642-25541-0_58.
- [5] Z. Haider et al., “Quantum Random Number Generator (QRNG): theoretical and experimental investigations,” *The European Physical Journal Plus*, vol. 138, no. 9, Sep. 2023, *doi*: <https://doi.org/10.1140/epjp/s13360-023-04421-3>.
- [6] U. Ansari, A. K. Chaudhary, and S. Verma, “Enhanced True Random Number Generator (TRNG) using Sensors for IoT Security Applications,” *2022 Third International Conference on Intelligent Computing Instrumentation and Control Technologies (ICI-CICT)*, pp. 1593–1597, Aug. 2022, *doi*: <https://doi.org/10.1109/icicict54557.2022.9917919>.
- [7] S. Hedayatpour and S. Chuprat, “Hash functions-based random number generator with image data source,” *IEEE*, 2025, *doi*: <https://doi.org/10.1109/ICOS.2011.6079248>.
- [8] C.-C. Liu, C.-C. Chen, J.-J. Yan, and T.-L. Liao, “Design of a Multiple Pseudorandom Number Generator Combined Chaotic System With RNS and Its Application to Secure Image Processing,” *IEEE Access*, vol. 12, pp. 155246–155258, 2024, *doi*: <https://doi.org/10.1109/access.2024.3482012>.
- [9] T. Nardi, “Generating Random Numbers With A Fish Tank,” Hackaday, Dec. 10, 2019. https://hackaday.com/2019/12/09/generating-random-numbers-with-a-fishtank/?utm_source=chatgpt.com (accessed Apr. 11, 2025).

- [10] G. Wirth, Pedro, and da Silva, “Random Telegraph Noise based True Random Number Generator for Fully Integrated Systems,” arXiv.org, 2023. <https://arxiv.org/abs/2301.05665> (accessed Apr. 11, 2025).
- [11] A. Stanco, D. G. Marangon, G. Vallone, S. Burri, E. Charbon, and P. Villorresi, “Efficient random number generation techniques for CMOS single-photon avalanche diode array exploiting fast time tagging units,” *Physical Review Research*, vol. 2, no. 2, Jun. 2020, doi: <https://doi.org/10.1103/physrevresearch.2.023287>.
- [12] X. WANG, R. ZHANG, and H. SHINOHARA, “A Single-Inverter-Based True Random Number Generator with On-Chip Clock-Tuning-Based Entropy Calibration Circuit,” *IEICE Transactions on Fundamentals of Electronics Communications and Computer Sciences*, vol. E107.A, no. 1, pp. 105–113, Jan. 2024, doi: <https://doi.org/10.1587/transfun.2023kep0011>.
- [13] N. Nguyen, Georges Kaddoum, Fabio Pareschi, Riccardo Rovatti, and G. Setti, “A fully CMOS true random number generator based on hidden attractor hyperchaotic system,” *Nonlinear Dynamics*, vol. 102, no. 4, pp. 2887–2904, Oct. 2020, doi: <https://doi.org/10.1007/s11071-020-06017-3>.
- [14] Byung Kwon Park et al., “Practical True Random Number Generator Using CMOS Image Sensor Dark Noise,” *IEEE Access*, vol. 7, pp. 91407–91413, Jan. 2019, doi: <https://doi.org/10.1109/access.2019.2926825>.
- [15] R. Katyal, A. Mishra, and A. Baluni, “True Random Number Generator using Fish Tank Image,” *International Journal of Computer Applications*, vol. 78, no. 16, pp. 38–40, Sep. 2013, doi: <https://doi.org/10.5120/13609-1419>.